DIPLOMA ECE / ELECTRONICS - LAB HANDBOOK

Fundamentals of C Programming Lab

A practical, screen-filling handbook for Course Code 3049. This is built like a lab manual plus textbook: concepts first, program patterns next, experiments with algorithms, common errors, viva questions, project ideas and exam preparation.

COURSE CODE
3049

CREDITS
1.5

SEMESTER
3

PERIODS
45 Practical Hours

C is not typing code. C is exact thinking.

Every program in this course must be written with input, processing, output, testing and record writing. In electronics work, this foundation later connects to embedded C, microcontrollers, HMI utilities, data logging and test automation.

Correct lab method: never jump directly to typing code.

SYLLABUS STRUCTURE

Official Course Map

The course is a practical lab subject. It has no lecture period, so the handbook is arranged around hands-on programming outcomes.

Course objectives

- ✓ Use C programming fundamentals confidently.
- ✓ Build a strong base for other programming languages and embedded C.
- ✓ Convert simple problem statements into algorithms and programs.
- ✓ Prepare clean lab records with source code, output and result.

> □□□ theory subject □□□□ □□□□□□□□□ □□□□□□ □□□. □□□ concept-□□□ program type □□□□□□ run □□□□□□ output □□□□□□□□□□□□□.

Prerequisite knowledge

- ✓ Basic mathematical operations and equations.
- ✓ Computer basics, operating system use and file handling.
- ✓ Previous exposure to Python programming is useful.
- ✓ Logical thinking: input - process - output.

Course outcomes and duration

CO	Outcome	Hours	Level	Handbook focus
CO 1	Make use of data types, I/O statements and basic operators.	11	Applying	printf, scanf, int, float, char, arithmetic, relational and logical operators.
CO 2	Apply decision making and loop statements.	13	Applying	if, if-else, nested if, switch, while, do while, for loop.
CO 3	Develop C programs based on arrays.	9	Applying	1D array, 2D array, sorting, matrix addition, subtraction, transpose and multiplication.
CO 4	Develop C programs using pointers, strings and user-defined functions.	9	Applying	Pointers, string functions, modular programming and reusable functions.
Tests	Series tests	3	-	Practical speed, correctness and viva.

CO-PO mapping

Outcome	PO1	PO2	PO3	PO4	PO5	PO6	PO7
CO1	3						
CO2	3		3	3			
CO3	3	3	3	3			
CO4	3	3	3	3			

Lab record format

1. Aim
2. Algorithm
3. Flowchart or logic explanation
4. Source program
5. Sample input and output
6. Result and viva notes

BEFORE CODING

Compiler, File and Lab Workflow

Recommended workflow

1. Create a folder for the experiment number.
2. Save file with meaningful name, for example `exp01_calculator.c`.
3. Compile first, then run.
4. Test minimum three inputs.
5. Copy final code and output to record.

Program skeleton

```
#include <stdio.h>
int main(void)
{
    /* declarations */
    /* input */
    /* processing */
    /* output */
    return 0;
}
```

Do not do this

- Do not memorize code without knowing logic.
- Do not ignore compiler warnings.
- Do not use variable names like `a,b,c` in big programs.
- Do not submit output without testing boundary cases.

Core rules of C that beginners forget

Rule	Meaning	Example	Common mistake
Every statement ends with semicolon	C compiler expects statement termination.	`sum = a + b;`	Missing semicolon after assignment.
Use correct format specifier	`scanf` and `printf` must match data type.	`%d` for int, `%f` for float, `%c` for char	Using `%d` for float.
= and == are different	= assigns, == compares.	`if(x == 5)`	`if(x = 5)`
Array index starts at 0	For size 5, valid indexes are 0 to 4.	`a[0]`, `a[4]`	Accessing `a[5]`.
String needs null character	C string ends with `'\0'`.	`char name[20];`	Insufficient array size for string.

MODULE 1 - CO1

Data Types, I/O Statements and Operators

Duration: 11 hours. Outcome: make use of data types, input/output statements and operators.

Input and output

`printf()` sends output to screen. `scanf()` reads input from keyboard.

```
int age;
printf("Enter age: ");
scanf("%d", &age);
printf("Age = %d", age);
```

Important: In `scanf`, use & for normal variables. For strings, the array name already acts as address.

Fundamental data types

Type	Use	Format
int	Whole numbers	%d
float	Decimal values	%f
double	Large decimal precision	%lf
char	Single character	%c

□□□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□□ data type □□□□□□□□□□□□□□□□□□□□.
Mark, voltage, length □□□□□□□□□□□□□□□□ □□□□□□□□□□ float □□□□.

Operators

- Arithmetic: `+ - * / %`
- Relational: `< > <= >= == !=`
- Logical: `&& || !`
- Assignment: `= += -= *= /=`
- Increment: ++, decrement: `--`

```
volume = height * width * depth
```

Solved program 1: arithmetic operations

```
#include <stdio.h>
int main(void)
{
    float a, b;
```

```
    printf("Enter two numbers: ");
    scanf("%f%f", &a, &b);
    printf("Addition = %.2f\n", a + b);
    printf("Subtraction = %.2f\n", a - b);
    printf("Multiplication = %.2f\n", a * b);
    if (b != 0) printf("Division = %.2f\n", a / b);
    else printf("Division not possible because second number is zero\n");
    return 0;
}
```

Solved program 2: quadratic equation roots

```
#include <stdio.h>
#include <math.h>
int main(void)
{
    float a, b, c, d, r1, r2;
    printf("Enter a, b and c: ");
    scanf("%f%f%f", &a, &b, &c);
    d = b * b - 4 * a * c;
    if (d > 0) {
        r1 = (-b + sqrt(d)) / (2 * a);
        r2 = (-b - sqrt(d)) / (2 * a);
        printf("Real and different roots: %.2f %.2f", r1, r2);
    } else if (d == 0) {
        r1 = -b / (2 * a);
        printf("Real and equal roots: %.2f", r1);
    } else printf("Roots are imaginary");
    return 0;
}
```

Solved program 3: cube volume

```
#include <stdio.h>
int main(void)
{
    float height, width, depth, volume;
    printf("Enter height, width and depth: ");
    scanf("%f%f%f", &height, &width, &depth);
    volume = height * width * depth;
    printf("Volume of cube/cuboid = %.2f", volume);
    return 0;
}
```

Test input: 2, 3, 4 should give 24.

MODULE 2 - CO2

Decision Making and Loop Statements

Duration: 13 hours. This is the logic-building module. Most lab mistakes happen here.

Decision statements

- `if` - one condition
- `if else` - two alternatives
- `nested if` - condition inside condition
- `switch` - selection from fixed choices

```
if (mark >= 50)
    printf("Pass");
else
    printf("Fail");
```

Loop statements

- `while` - entry-controlled loop
- `do while` - exit-controlled loop
- `for` - compact loop with initialization, condition and increment

```
for (i = 1; i <= 10; i++)
    printf("%d\n", i);
```

Flowchart thinking

Condition decides the execution path. Do not write nested logic blindly. Trace with sample input first.

Largest, smallest and even/odd

```
#include <stdio.h>
int main(void)
{
    int a, b, c, largest, smallest;
    printf("Enter three numbers: ");
    scanf("%d%d%d", &a, &b, &c);
    largest = a;
    if (b > largest) largest = b;
```

```
    if (c > largest) largest = c;
    smallest = a;
    if (b < smallest) smallest = b;
    if (c < smallest) smallest = c;
    printf("Largest = %d", largest);
    printf("\nSmallest = %d", smallest);
    printf("\nLargest is %s", (largest % 2 == 0) ? "even" : "odd");
    printf("\nSmallest is %s", (smallest % 2 == 0) ? "even" : "odd");
    return 0;
}
```

Vowel or consonant using switch

```
#include <stdio.h>
int main(void)
{
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);
    switch (ch) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
        case 'A': case 'E': case 'I': case 'O': case 'U':
            printf("Vowel"); break;
        default:
            printf("Consonant or non-vowel character");
    }
    return 0;
}
```

Notice: Space before `%c` in `scanf(" %c", &ch)` avoids reading leftover newline.

Fibonacci series

```
#include <stdio.h>
int main(void)
{
    int n, i, a = 0, b = 1, next;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    printf("Fibonacci series: ");
    for (i = 1; i <= n; i++) {
        printf("%d ", a);
        next = a + b;
        a = b;
        b = next;
    }
```

```
    return 0;
}
```

MODULE 3 - CO3

Arrays and Matrix Programs

Duration: 9 hours. Arrays are used when many values of the same type must be stored and processed together.

1D array concept

An array stores multiple values under one name.

```
int mark[5];
mark[0] = 75;
mark[1] = 80;
```

> Array index □□□□□□□□ 0 □□□□ □□□□□□□□. Size 5 □□□□□□□□ □□□□□ index 4 □□□.

2D array concept

2D array is useful for matrix operations.

```
int a[3][3];
scanf("%d", &a[i][j]);
```

Array operation checklist

- ✓ Read size.
- ✓ Read elements using loop.
- ✓ Process using loop.
- ✓ Display result clearly.
- ✓ Never access outside limit.

Sum and reverse display of array

```c
#include <stdio.h>
int main(void)
{
    int a[50], n, i, sum = 0;
    printf("Enter size: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) { scanf("%d", &a[i]); sum += a[i]; }
    printf("Sum = %d\n", sum);
    printf("Reverse order: ");
    for (i = n - 1; i >= 0; i--) printf("%d ", a[i]);
    return 0;
}
```

Ascending sort, largest and second largest

```c
#include <stdio.h>
int main(void)
{
    int a[50], n, i, j, temp;
    printf("Enter size: "); scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (a[i] > a[j]) { temp = a[i]; a[i] = a[j]; a[j] = temp; }
    printf("Sorted array: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\nLargest = %d", a[n - 1]);
    printf("\nSecond largest = %d", a[n - 2]);
    return 0;
}
```

Matrix multiplication after checking condition

```c
#include <stdio.h>
int main(void)
{
    int a[10][10], b[10][10], c[10][10];
    int r1, c1, r2, c2, i, j, k;
    printf("Enter rows and columns of first matrix: "); scanf("%d%d", &r1,
&c1);
    printf("Enter rows and columns of second matrix: "); scanf("%d%d", &r2,
&c2);
    if (c1 != r2) { printf("Matrix multiplication not possible"); return 0;
}
    printf("Enter first matrix:\n");
```

```
    for (i = 0; i < r1; i++) for (j = 0; j < c1; j++) scanf("%d", &a[i]
[j]);
    printf("Enter second matrix:\n");
    for (i = 0; i < r2; i++) for (j = 0; j < c2; j++) scanf("%d", &b[i]
[j]);
    for (i = 0; i < r1; i++) for (j = 0; j < c2; j++) {
        c[i][j] = 0;
        for (k = 0; k < c1; k++) c[i][j] += a[i][k] * b[k][j];
    }
    printf("Product matrix:\n");
    for (i = 0; i < r1; i++) { for (j = 0; j < c2; j++) printf("%d\t", c[i]
[j]); printf("\n"); }
    return 0;
}
```

MODULE 4 - CO4

Pointers, Strings and User-defined Functions

Duration: 9 hours. This module prepares students for embedded C and modular problem solving.

Pointer basics

A pointer stores the address of another variable.

```
int x = 10;
int *p;
p = &x;
printf("%d", *p);
```

& means address of. ***** means value at address when used with pointer variable.

String functions

Function	Use
strlen()	Find length
strcpy()	Copy string
strcat()	Join strings
strcmp()	Compare strings

User-defined functions

Functions reduce repetition and make programs readable.

```
int add(int a, int b)
{
    return a + b;
}
```

Biggest among three numbers using pointer

```
#include <stdio.h>
int main(void)
{
    int a, b, c, *p1, *p2, *p3, biggest;
    printf("Enter three numbers: "); scanf("%d%d%d", &a, &b, &c);
    p1 = &a; p2 = &b; p3 = &c;
    biggest = *p1;
    if (*p2 > biggest) biggest = *p2;
    if (*p3 > biggest) biggest = *p3;
    printf("Biggest = %d", biggest);
    return 0;
}
```

Concatenate two strings using built-in function

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char first[80], second[40];
    printf("Enter first string: "); scanf("%s", first);
    printf("Enter second string: "); scanf("%s", second);
    strcat(first, second);
    printf("Concatenated string = %s", first);
    return 0;
}
```

Matrix sums using user-defined functions

```
#include <stdio.h>
int diagonalSum(int a[10][10], int n)
{
    int i, sum = 0;
    for (i = 0; i < n; i++) sum += a[i][i];
```

```
    return sum;
}
int main(void)
{
    int a[10][10], r, c, i, j, total = 0;
    printf("Enter rows and columns: "); scanf("%d%d", &r, &c);
    printf("Enter matrix elements:\n");
    for (i = 0; i < r; i++) for (j = 0; j < c; j++) { scanf("%d", &a[i]
[j]); total += a[i][j]; }
    printf("Total sum = %d\n", total);
    if (r == c) printf("Diagonal sum = %d\n", diagonalSum(a, r));
    for (j = 0; j < c; j++) {
        int colsum = 0;
        for (i = 0; i < r; i++) colsum += a[i][j];
        printf("Column %d sum = %d\n", j + 1, colsum);
    }
    return 0;
}
```

LAB MANUAL

Suggested Experiments and Additional Practice

Use this section as a practical checklist. Each experiment should be run, tested and recorded.

Official suggested experiments

1. Arithmetic operations on two numbers.
2. Roots of quadratic equation.
3. Volume of cube/cuboid.
4. Largest and smallest among three numbers; check even/odd.
5. Vowel or consonant using switch.
6. Fibonacci series.
7. Array sum and reverse display.
8. Sort array, largest and second largest.
9. Matrix multiplication after checking condition.
10. Biggest among three numbers using pointer.
11. String concatenation using built-in functions.
12. Matrix sums using user-defined functions.

Extra practice programs for mastery

1. Simple interest and compound interest.
2. Electricity bill using slab rates.
3. Student mark sheet with grade.

4. Prime number check.
5. Factorial using loop.
6. Palindrome number and palindrome string.
7. Count vowels, consonants, digits and spaces in a string.
8. Linear search and binary search.
9. Transpose of matrix.
10. Menu-driven calculator using switch.

Lab evaluation checklist

Item	What examiner checks	Common failure	Fix
Aim	Clear problem statement.	Aim copied wrongly.	Write exactly what the program should do.
Algorithm	Logical steps before code.	Algorithm missing input/processing/output.	Use numbered steps.
Code	Compiles without error.	Missing braces, format mismatch.	Compile repeatedly during writing.
Output	Shows input and result.	Only final result copied.	Take output with sample inputs.
Viva	Can explain code.	Memorized without logic.	Trace the program line by line.

DEBUGGING

Common Errors and Practical Fixes

Compile-time errors

Error	Reason	Fix
Expected ';'	Semicolon missing.	Check previous line also.
Undeclared identifier	Variable not declared or spelling mismatch.	Declare variable before use.
Stray character	Copied smart quotes or hidden characters.	Retype the line manually.
Undefined reference to sqrt	Math library not linked in some compilers.	Use correct compiler setting or link math library.

Logical errors

- Wrong condition: using > instead of >=.

- Wrong loop limit: using `i <= n` for array size n.
- Integer division when decimal answer is required.
- Forgetting to initialize sum/product variables.
- Using uninitialized array elements.

Debugging method that actually works

1. Read the compiler error from top, not bottom.
2. Fix only the first error first.
3. Print intermediate values using temporary `printf`.
4. Test with small input where you know the answer manually.
5. Remove temporary debug print before record submission.

OPEN-ENDED WORK

Mini Project Ideas

The syllabus suggests open-ended projects for continuous evaluation. These projects should be small but complete.

Student CGPA calculator

Use arrays for marks or grade points. Use functions for grade conversion and CGPA calculation.

- Input: number of subjects, marks or grade points.
- Output: total, grade, CGPA.
- Upgrade: store student names using strings.

Consumer billing

Useful for electrical/electronics students. Apply slab-based billing using if-else.

- Input: consumer number, units.
- Processing: calculate charge by slab.
- Output: formatted bill.

Candidate list sorting

Use arrays and strings. Sort candidate marks or names.

- Input: names and marks.
- Processing: sort in descending mark order.
- Output: rank list.

Electronics-oriented extra project ideas

- Resistor color code calculator.
- LED series resistor calculator.
- Battery backup time calculator.
- Ohm's law calculator.
- Panel wire ferrule list sorter.
- Production inspection defect counter.

> □□ □□□□□□□□□□□ C programming lab □□□ real engineering utility □□□ □□□□□. □□□□□ sample program □□□ □□□□□□□.

VIVA

Frequently Asked Viva Questions

Basic viva

Q1 Why is `#include <stdio.h>` used?

Q2 What is the difference between `printf` and `scanf`?

Q3 Why is & used in `scanf`?

Q4 What is the difference between = and ==?

Q5 What is the use of `return 0`?

Q6 What is a format specifier?

Advanced lab viva

Q7 What is an array and why is it used?

Q8 What is a pointer?

Q9 What is the difference between call by value and call by reference?

Q10 Why does a C string end with `\0`?

Q11 What is nested loop?

Q12 How is matrix multiplication condition checked?

PRACTICAL EXAM

Model Practical Test

Model Lab Examination - 3 Hours

1. Write algorithm and C program to read two matrices and print their sum and transpose of the first matrix.
2. Write a C program to read a string and check whether it is a palindrome. Also count vowels.
3. Write a user-defined function to find the largest and smallest elements in an integer array.
4. Viva: Explain pointer, array indexing, string termination and difference between while and do-while.
5. Record verification: submit aim, algorithm, source code, output and result.

Mark split suggestion

Component	Marks
Algorithm / logic	20
Correct source program	30
Execution and output	25
Viva	15
Record	10

Practical exam warnings

- Do not start with long code. First write logic.
- Use small test data.

- Do not change multiple lines blindly when error occurs.
- Check brackets and semicolons before asking for help.

ANSWER KEY

Short Answers for Viva

Core viva answers

stdio.h: Header file that declares standard input and output functions such as printf and scanf.

printf: Displays output. **scanf:** Reads input.

& in scanf: Gives the address of the variable so that scanf can store the input value there.

= and ==: = is assignment. == is comparison.

Array: Collection of same type elements stored in continuous memory locations.

Pointer: Variable that stores the address of another variable.

String: Character array terminated by null character.

Function: Named block of reusable code.

Matrix multiplication condition: Columns of first matrix must be equal to rows of second matrix.

Quick revision

- ✓ Program starts execution from `main()`.
- ✓ Use `%d` for int, `%f` for float, `%c` for char.
- ✓ Use loop for repeated operations.
- ✓ Use array for multiple same-type values.
- ✓ Use function for repeated logic.

One-day exam preparation

1. Revise syntax and format specifiers.
2. Practice arithmetic, if-else and switch.
3. Practice Fibonacci and factorial.

4. Practice array sort and matrix operations.
5. Practice pointer, string and function programs.

SOURCES

Text and Online References

Text / Reference

- Let Us C - Yashavant Kanetkar, BPB Publications.
- Programming in ANSI C - E. Balagurusamy, Tata McGraw Hill.
- A Text Book on C - E. Karthikeyan, PHI Learning Pvt. Ltd.
- Programming in C - D. Ravichandran, New Age International Publishers.
- The C Programming Language - Brian W. Kernighan and Dennis M. Ritchie, Prentice Hall.

Online learning links from syllabus

- freecomputerbooks.com - The C Programming Language.
- TutorialsPoint C Programming tutorial.
- Guru99 C programming tutorial.
- cprogramming.com.
- Zentut C tutorial.

Final professional note

For electronics and escalator-related engineering work, C programming is not just an academic lab. The same thinking is used in embedded controllers, test jigs, serial utilities, inspection tools, data converters and automation scripts. Learn to write clean, testable logic now; otherwise embedded C later will become painful.